EPITECH.
PARIS GRADUATE SCHOOL OF DIGITAL INNOVATION

KubeQuest

Kick-off

T10 - Cloud

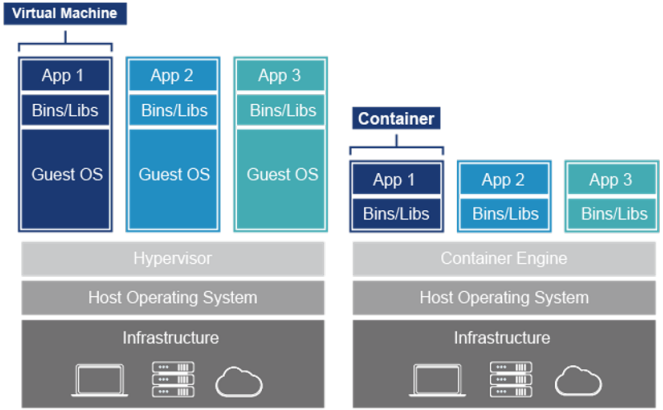
T-CLO-902

The rise of Docker

Architecture:

- Abstraction layer between OS and application
- Package runtime (ex: NodeJS, libs…) + application into a single container
- Basically a tar.gz

The rise of Docker



The rise of Docker

Use case:

- 2 apps: first in NodeJS v12 + second in NodeJS v14
- Start a Redis instance in 10s without any install
- Ensure you run the same thing in staging and production
- Multiple containers per VM == cheaper
- Build once, run everywhere

The rise of Docker

Isolation at kernel level:

- 1 process isolated from neighborhood
- Kernel feature: *namespace* and *cgroups*
- Process don't see each other

Shared kernel:

- Some bottlenecks
- Shared resources
- Security (!)

The rise of Docker

Microservices and Docker containers both grew at the same time.

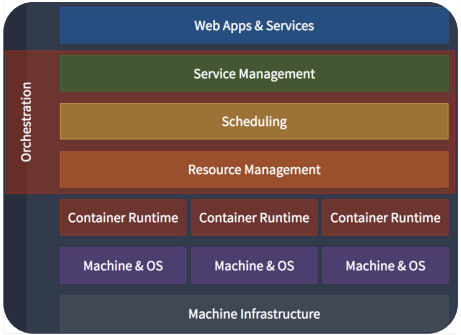
There is no microservices if infrastructure cannot support it.

The pride of Docker

Docker has been built by **Epitech** alumni.

It means a lot ;)

Container orchestrator



Abstraction between

a cluster of virtual machines

and some containers.

Orchestrator tools

Apache MESOS

OPENSHIFT

HashiCorp Nomad

RANCHER

What about the sysadmin job ?

3 levels:

- infrastructure (hardware) <- cloud provider
- platform (orchestrator) <- system administrator
- application (container) <- developer

Kubernetes scheduling constraints

- Start X instances of a service
- Spread in multiple bare-metal or data centers
- Or ensure service A is on the same server as service B
- Resource management (reserve CPU, memory, disk…)
- Auto-scaling of services
- Auto-scaling of nodes
- Bin packing
- Node drain

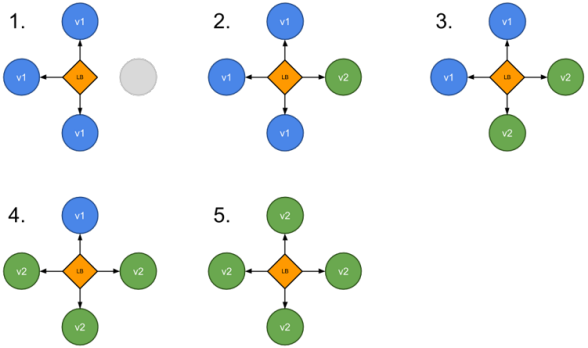
Kubernetes lifecycle constraints

- Start service A after service B's job is done (pre-start)
- Start twice a day, every 12 hours
- Restart on failure
- Health-check

Kubernetes deployment constraints

- Zero-downtime deployment
- Auto-healing
- 1-click rollback
- Blue-green, canary or rolling-update

Kubernetes illustrated



Kubernetes illustrated

Kubernetes

ACL

a user or application must have permission
before carrying out an operation in a k8s cluster

You should see K8s as an infrastructure framework

Kubernetes cool features

- Internal DNS
- Volumes
- Secret management
- Multi-cloud & cross region

Kubernetes limits

- Stateful services (ex: databases)
- K8s on top of IaaS can be a duplicate
- K8s is a war machine, hard to set up
- A lot of config files
- Multi-tenant k8s cluster is a security issue

Kubernetes

How to run a k8s cluster?

Amazon EKS

Azure Kubernetes Service (AKS)

K3S

minikube

Any questions

?